

Browser developer tools:

Leverage the browser's developer tools, such as the JavaScript console, network tab, and debugger. These tools allow you to inspect network requests, log JavaScript errors, step through code execution, and examine the state and structure of the application's DOM.

Code review: Have peers or team members review your code. Code reviews can help identify potential issues, provide valuable feedback, and ensure code quality.

User testing: Involve real users or testers in the process to gather feedback and identify potential issues

from a user's perspective. User testing can uncover usability problems, edge cases, or unexpected behaviour that may not have been detected during development.

By combining these testing and debugging approaches, developers can ensure the reliability, functionality, and user experience of their HyperApp applications. **END** 

References

- <https://www.digitalocean.com/community/tutorials/js-hyperapp>
- <https://fireflysemantics.medium.com/what-is-reactive-state-management-51dcef222023>

By: Dr Magesh Kasthuri and Dr Anand Nayyar

Dr Magesh Kasthuri is a senior distinguished member of the technical staff and principal consultant at Wipro Ltd. This article expresses his views and not that of Wipro.

Dr Anand Nayyar is a PhD in wireless sensor networks and swarm intelligence. He works at Duy Tan University, Vietnam, and loves to explore open source technologies, IoT, cloud computing, deep learning and cyber security.

Continued to page...87

Example: Saving data from Python's *BeautifulSoup* to a CSV file:

```
import re
import requests
from bs4 import BeautifulSoup as bs
import csv

# Making a GET request
r = requests.get('https://flask-bulma-css.appseed.us/')

# Parsing the HTML
htmlObj = bs(r.content, 'html.parser')

# Finding by id
s = htmlObj.find('div', id="cloned-navbar-menu")

# Getting the leftbar
a = s.find('a', class_="navbar-item is-hidden-mobile")

# All the li under the above ul
content = a.find_all('path')
lst = []
for i in range(3):
    dict = {}
    s = str(content[i])
    dict['path_name'] = re.search('class="path[1-3]"', s)
    dict['path_value'] = re.search('d=".*$', s)
    lst.append(dict)
filename = 'page.csv'
with open(filename, 'w', newline='') as f:
    w = csv.DictWriter(f,['path_name','path_value'])
```

```
w.writeheader()
w.writerows(lst)
f.close()
```

The output CSV file is given below.

path_name	path_value
<re.Match object; span=(6, 19), match='class="path1"'>	<re.Match object; span=(20, 111), match='d="M 300 400 L 700 400 C 900 400 900 750 600 850 >
<re.Match object; span=(6, 19), match='class="path2"'>	<re.Match object; span=(20, 51), match='d="M 300 500 L 700 500"></path>>
<re.Match object; span=(6, 19), match='class="path3"'>	<re.Match object; span=(20, 111), match='d="M 700 600 L 300 600 C 100 600 100 200 400 150 >

Web scraping with Python web scripting is a powerful tool for web data analysis. Here we have mentioned a few useful *requests* and *BeautifulSoup* library methods. Hope you will find the information useful and start your journey into this prevailing technology for data analytics of web information. **END** 

By: Dr Dipankar Ray

The author has more than 20 years of experience in software development. He has published two books on MATLAB and medical image processing (published by PHI Learning and SPD).